Cloud Computing Lab 13

Submitted To: Sir Waqas Saleem & Sie Shoaib
Submitted By: Hamail Fatima
Roll No: 2023-BSE-023
Section: 5(A)

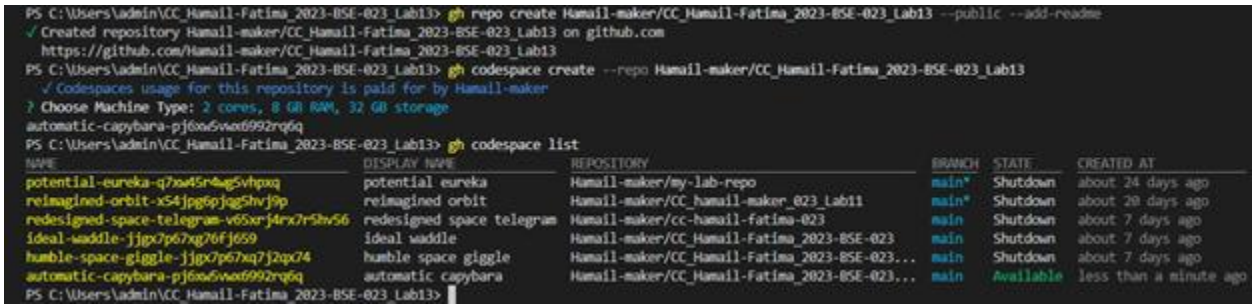
Course Name: Cloud Computing

Lab 12 repo link: https://github.com/Hamail-maker/CC_Hamail-Fatima_2023-BSE-023_Lab13.git

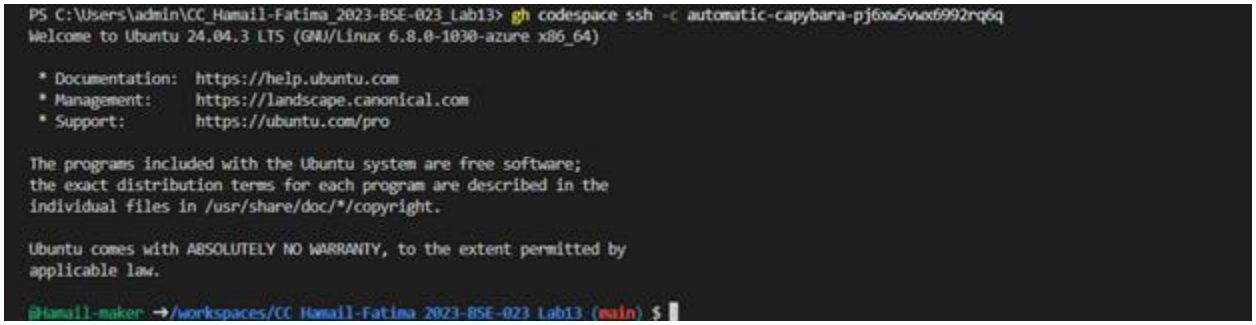
Lab # 13

Task 0 Lab Setup (Codespace & GH CLI)
All actions below should be executed inside the Codespace shell.
Create Codespace & connect:
create or open codespace via GH CLI (example)
gh repo create CC_<YourName>_<YourRollNumber>/Lab13 --public
gh codespace create --repo <user_name>/Lab13
gh codespace list
gh codespace ssh -c <your_codespace_name>

- **Save screenshot as:** task0_codespace_create_and_list.png — output showing repo creation/codespace list.



- **Save screenshot as:** task0_codespace_ssh_connected.png — terminal inside the Codespace shell after ssh.



Task 1 — Create IAM Group and Output Details
In this task, you will create an IAM group named "developers" and output its details.

1. Create the initial project structure:

mkdir -p ~/Lab13
cd ~/Lab13

- **Save screenshot as:** task1_project_directory.png — terminal showing directory creation.

2. Create the main Terraform file:

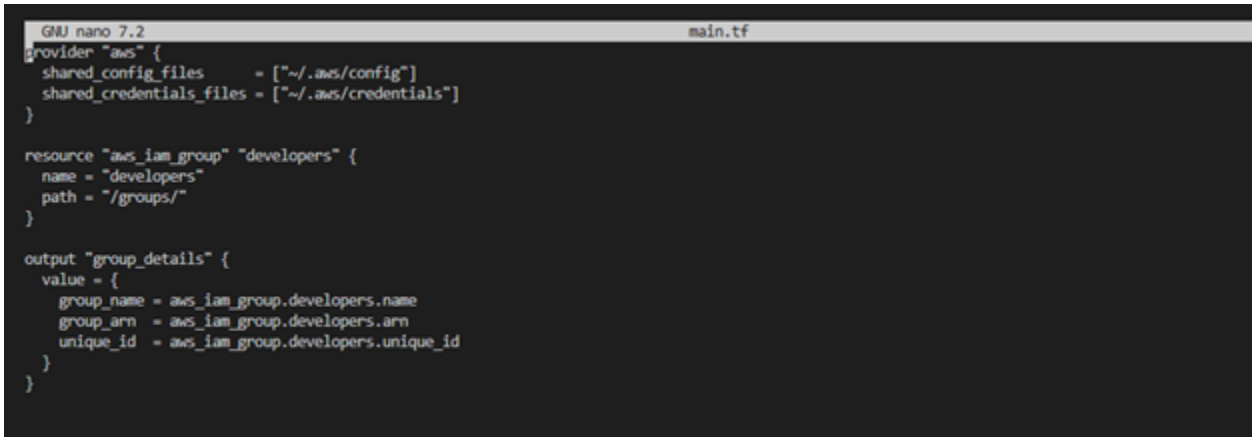
touch main.tf

- **Save screenshot as:** task1_file_created.png — terminal showing file creation.

3. Create main.tf with AWS provider configuration:

```
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
```

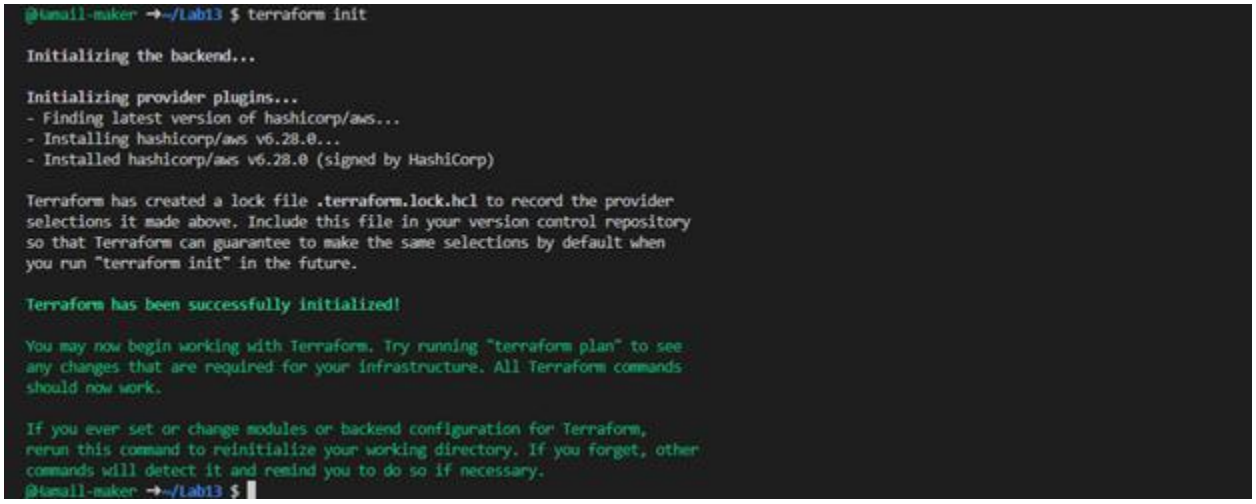
- **Save screenshot as:** task1_main_tf. png — content of main.tf file.



4. Initialize Terraform:

```
terraform init
```

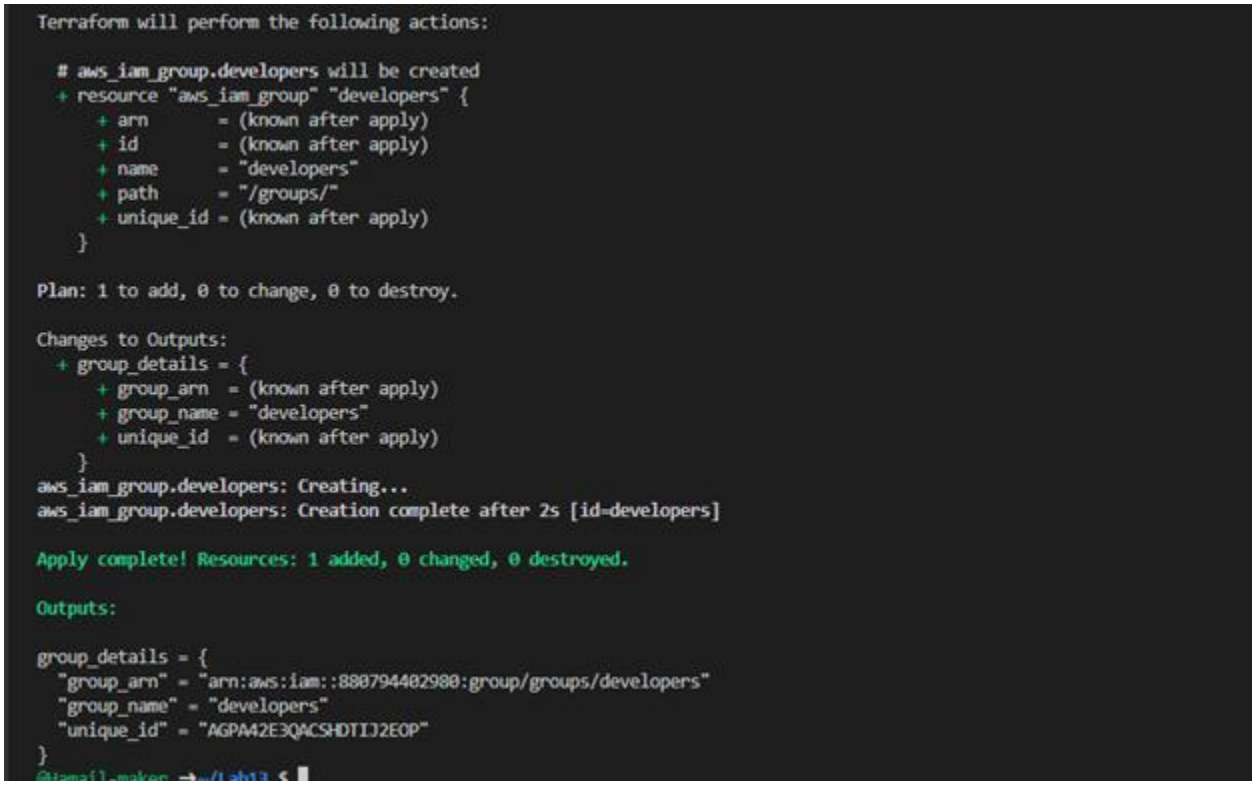
- **Save screenshot as:** task1_terraform_init.png — terraform init output.



5. Apply the configuration:

terraform apply -auto-approve

- **Save screenshot as:** task1_terraform_apply.png — terraform apply output showing group created.



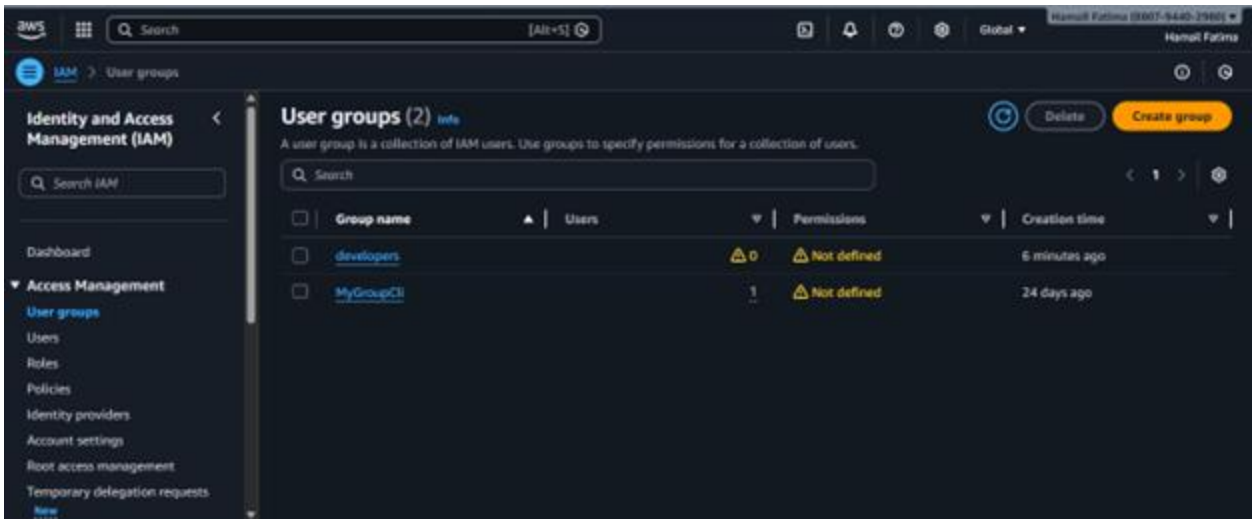
6. Display the output:

terraform output

- **Save screenshot as:** task1_terraform_output.png — terraform output showing group details.

7. Verify the group in AWS Console:

- Navigate to IAM → Groups in AWS Console
- **Save screenshot as:**



— AWS Console showing the developers group.

Task 2 — Create IAM User with Group Membership

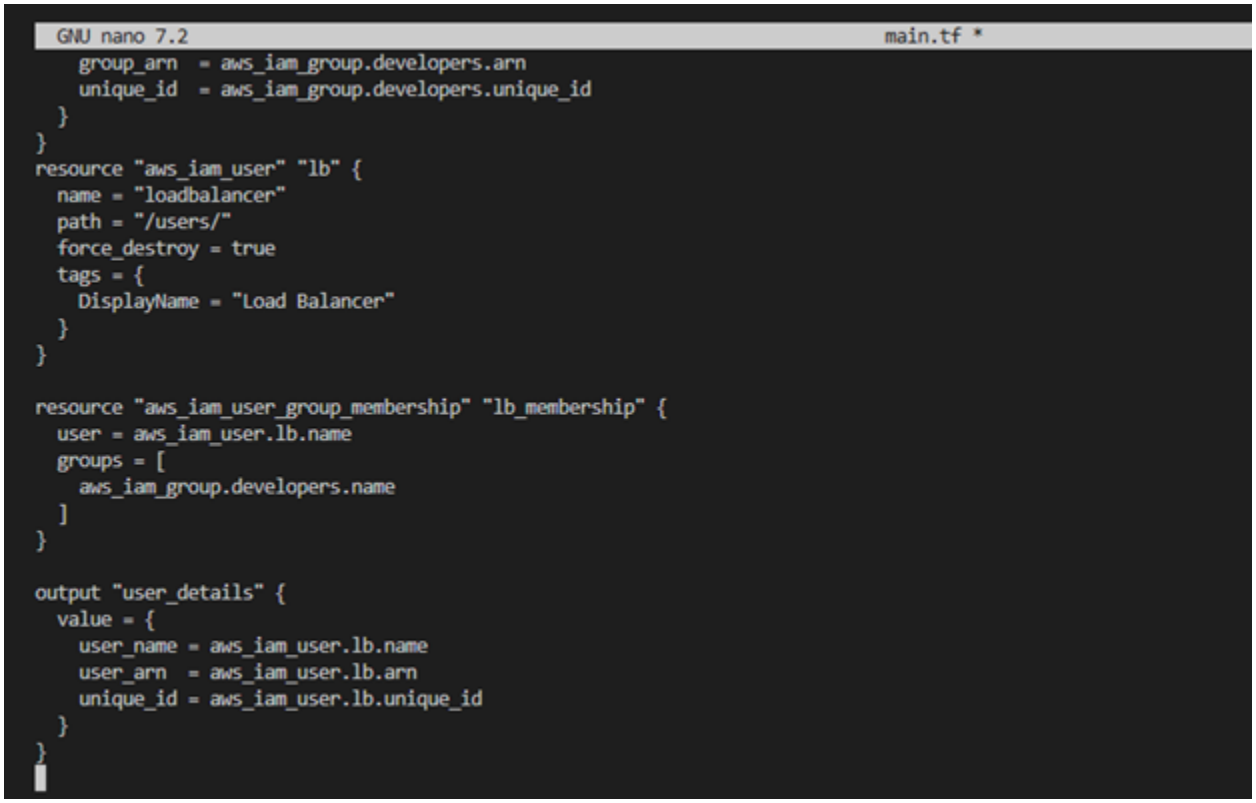
In this task, you will create an IAM user named "loadbalancer" and add it to the developers group.

1. Update main.tf to add the IAM user resource:

```
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers. name
    group_arn  = aws_iam_group. developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}

resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}

resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group. developers.name
  ]
}
```

```
output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}
```

- **Save screenshot as:**



— updated main.tf with user resources.

2. Apply the configuration:

```
terraform apply -auto-approve
```

- **Save screenshot as:** task2_terraform_apply.png — terraform apply output showing user created.

```
@Hamail-maker →~/Lab13 $ terraform apply -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.lb will be created
  + resource "aws_iam_user" "lb" {
      + arn           = (known after apply)
      + force_destroy = true
      + id            = (known after apply)
      + name          = "loadbalancer"
      + path          = "/users/"
      + tags          = {
          + "DisplayName" = "Load Balancer"
        }
      + tags_all      = {
          + "DisplayName" = "Load Balancer"
        }
      + unique_id     = (known after apply)
    }

  # aws_iam_user_group_membership.lb_membership will be created
  + resource "aws_iam_user_group_membership" "lb_membership" {
      + groups = [
          + "developers",
        ]
```

3. Display the outputs:

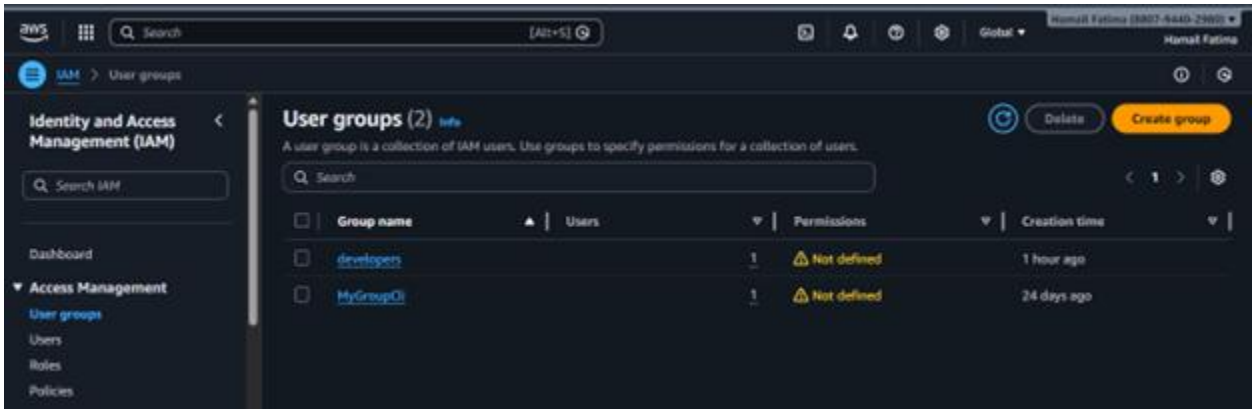
terraform output

- **Save screenshot as:** task2_terraform_output.png — terraform output showing user and group details.

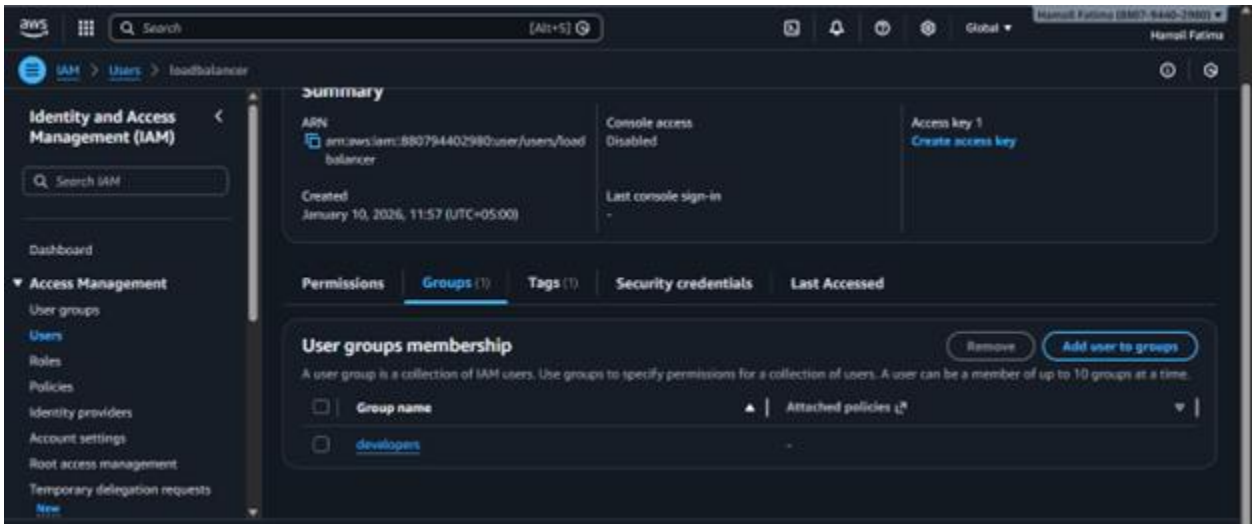
```
@Hamail-maker →~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
user_details = {
  "unique_id" = "AIDA42E3QACSL6WSGHQDT"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@Hamail-maker →~/Lab13 $
```

4. Verify the user in AWS Console:

- Navigate to IAM → Users in AWS Console
- Click on "loadbalancer" user
- Check the "Groups" tab
- **Save screenshot as:** task2_aws_console_user.png — AWS Console showing the loadbalancer user.



- **Save screenshot as:** task2_aws_console_user_groups.png — AWS Console showing user's group membership.



Task 3 — Attach Policies to IAM Group

In this task, you will attach AWS managed policies (AmazonEC2FullAccess and IAMUserChangePassword) to the developers group.

1. Update main.tf to add policy attachments:

```
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}

resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}

resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group.developers.name
  ]
```

```
}

output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}

resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
    group = aws_iam_group.developers.name
    policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
     group = aws_iam_group.developers.name
     policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}
```

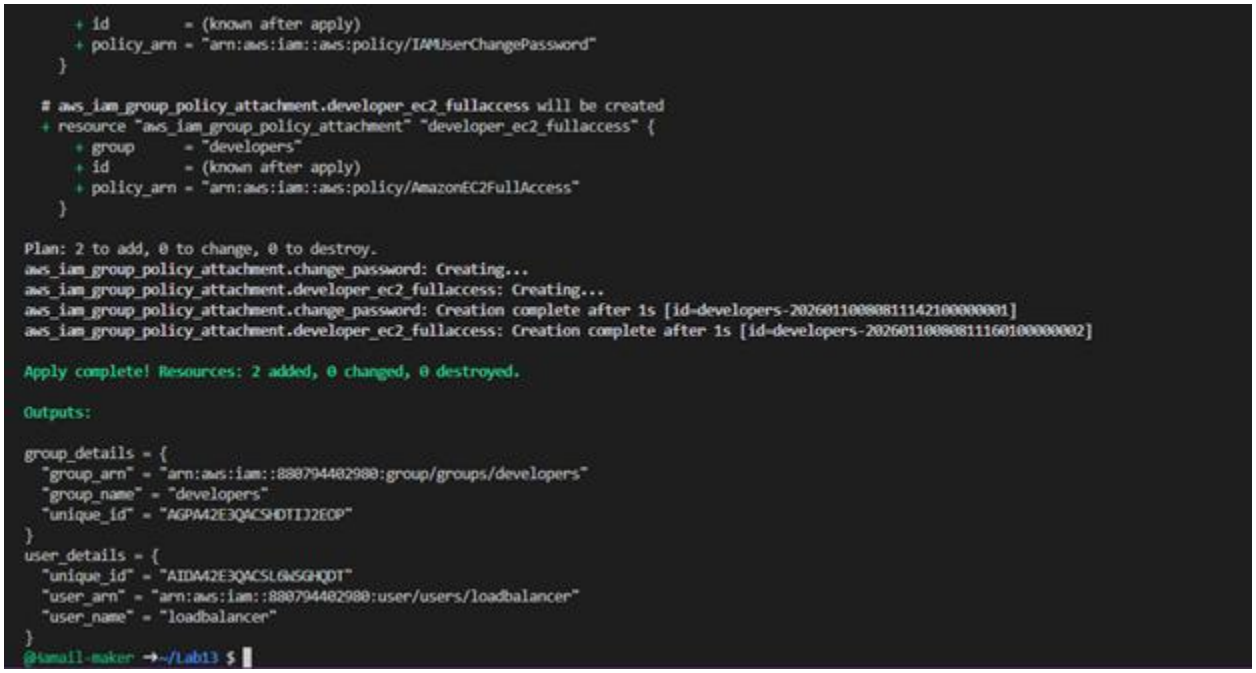
- **Save screenshot as:** \

— updated main.tf with policy attachments.

2. Apply the configuration:

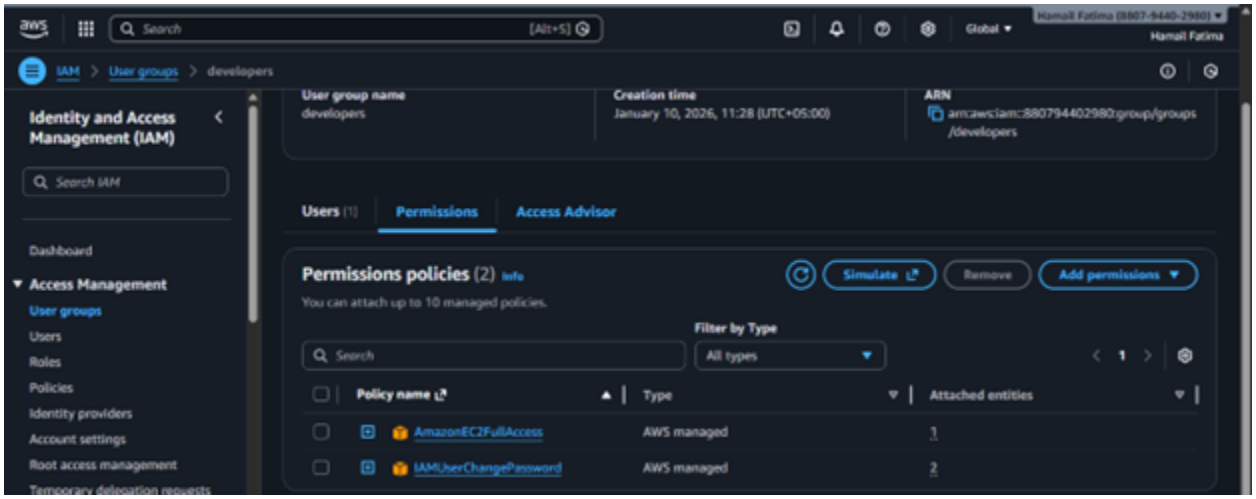
terraform apply -auto-approve

- **Save screenshot as:** task3_terraform_apply.png — terraform apply output showing policies attached.



3. Verify policies in AWS Console:

- Navigate to IAM → Groups → developers
- Click on "Permissions" tab
- **Save screenshot as:** task3_aws_console_policies.png — AWS Console showing attached policies.



Task 4 — Create Login Profile for IAM User

In this task, you will create a login profile for the loadbalancer user using a bash script and null_resource provisioner.

1. Create variables.tf file:

```
variable "iam_password" {
  description = "Temporary password for the IAM user"
  type        = string
  sensitive   = true
  default = "1dontKnow"
}
```

- **Save screenshot as:** task4_variables_tf.png — content of variables.tf file.

2. Create the bash script create-login-profile.sh:

```
#!/usr/bin/env bash
set -euo pipefail

USERNAME="$1"
PASSWORD="$2"

# Check if login profile already exists
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then
  echo "Login profile already exists for $USERNAME.  Skipping."
else
  echo "Creating login profile for $USERNAME"
  aws iam create-login-profile \
    --user-name "$USERNAME" \
    --password "$PASSWORD" \
    --password-reset-required
```

fi

- **Save screenshot as:** task4_create_login_script.png — content of create-login-profile.sh.

```
@Hamail-maker →~/Lab13 $ nano create-login-profile.sh
@Hamail-maker →~/Lab13 $ cat create-login-profile.sh
#!/usr/bin/env bash
set -euo pipefail

USERNAME="$1"
PASSWORD="$2"

# Check if login profile already exists
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then
  echo "Login profile already exists for $USERNAME. Skipping."
else
  echo "Creating login profile for $USERNAME"
  aws iam create-login-profile \
    --user-name "$USERNAME" \
    --password "$PASSWORD" \
    --password-reset-required
fi
@Hamail-maker →~/Lab13 $
```

3. Make the script executable:

chmod +x create-login-profile.sh

- **Save screenshot as:** task4_chmod_script.png — terminal showing chmod command.

```
@Hamail-maker →~/Lab13 $ chmod +x create-login-profile.sh
@Hamail-maker →~/Lab13 $
```

4. Update main.tf to add the null_resource provisioner:

Add this resource after the user creation:

```
resource "null_resource" "create_login_profile" {
  triggers = {
    password_hash = sha256(var.iam_password)
    user          = aws_iam_user.lb.name
  }

  depends_on = [aws_iam_user. lb]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${aws_iam_user.lb.name}
'${var.iam_password}'"
  }
}
```

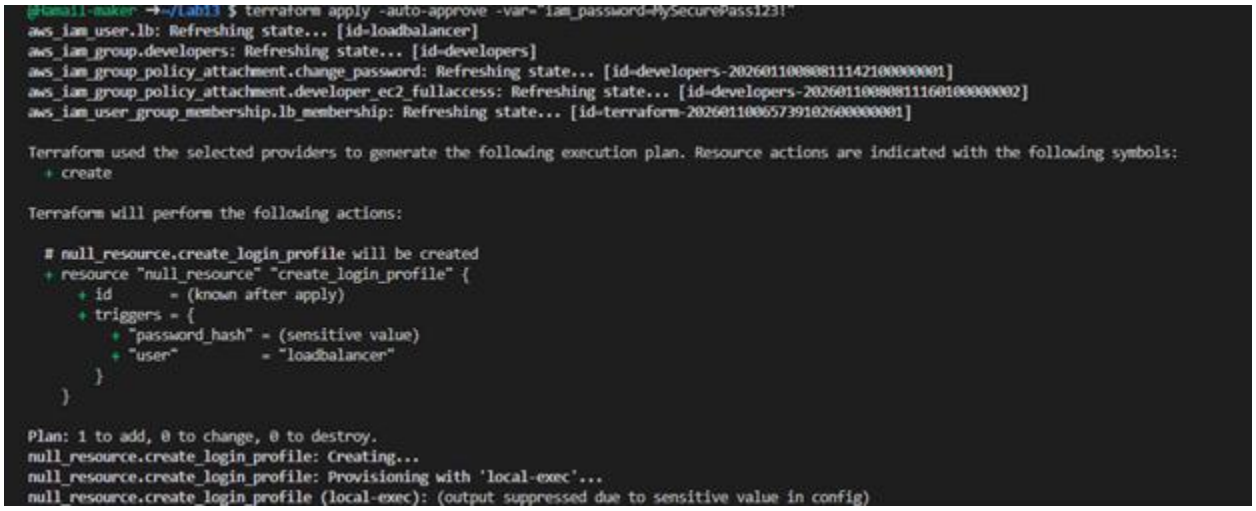
- **Save screenshot as:** task4_main_tf_login_profile.png — main.tf showing null_resource.

```
output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn  = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}
resource "null_resource" "create_login_profile" {
  triggers = {
    password_hash = sha256(var.iam_password)
    user          = aws_iam_user.lb.name
  }

  depends_on = [aws_iam_user.lb]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${aws_iam_user.lb.name} '${var.iam_password}'"
  }
}
```

5. Apply the configuration with a custom password:

terraform apply -auto-approve -var="iam_password=MySecurePass123!"

- **Save screenshot as:** task4_terraform_apply.png — terraform apply output showing login profile creation.



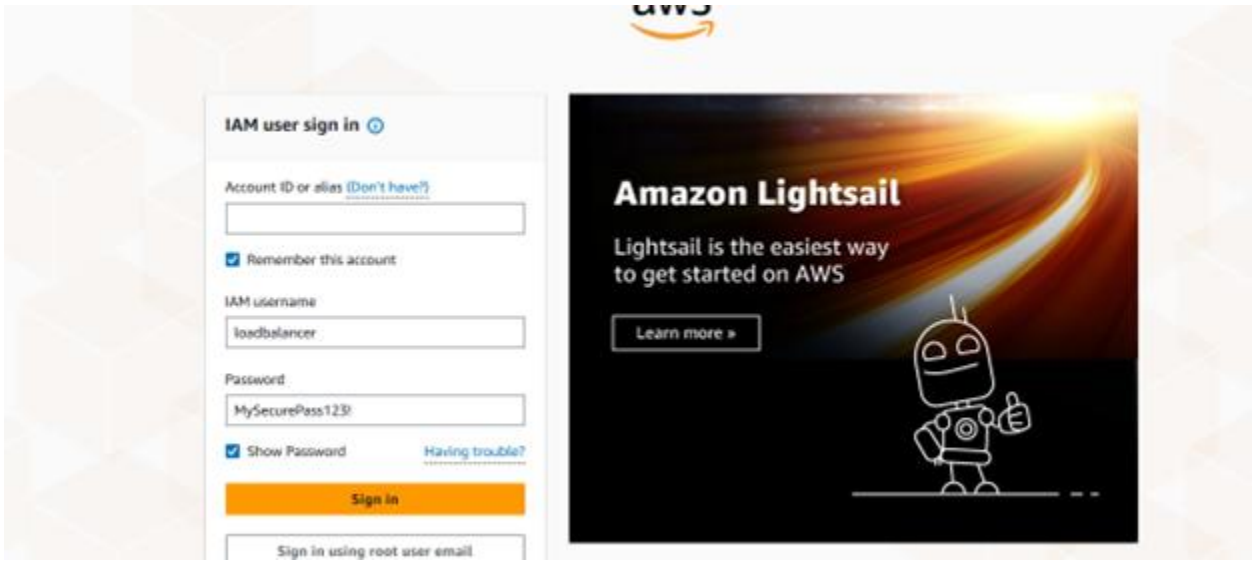
6. Verify login profile creation:

aws iam get-login-profile --user-name loadbalancer

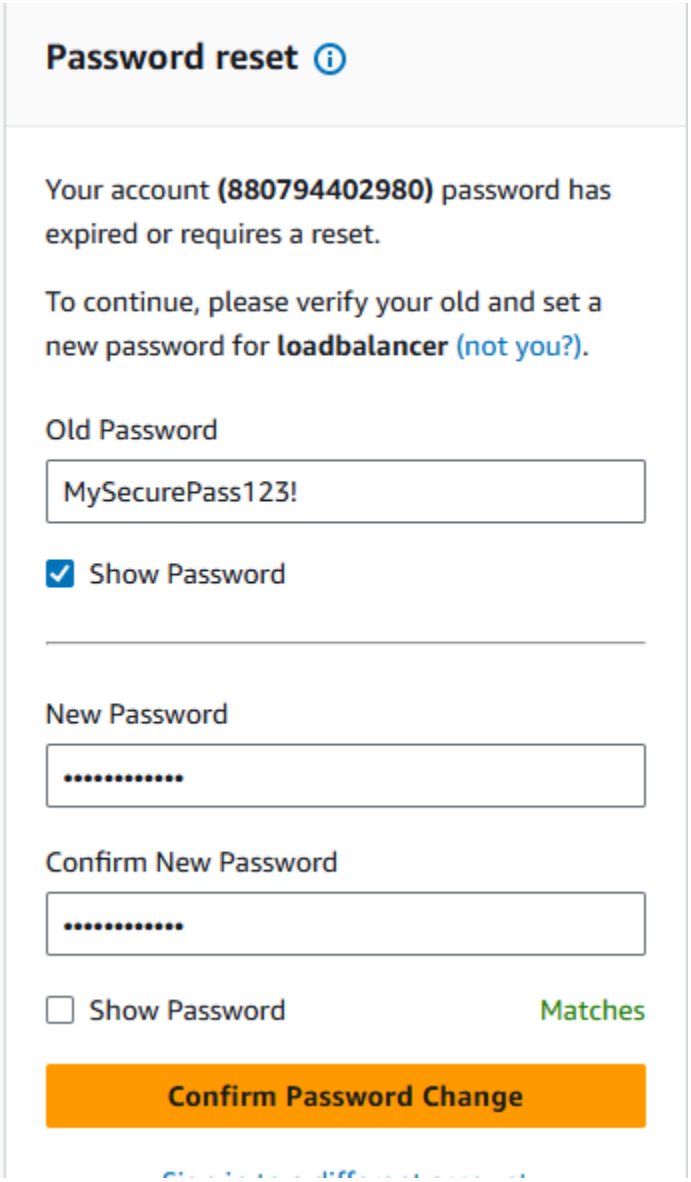
- **Save screenshot as:** task4_aws_cli_verify.png — AWS CLI output showing login profile.

7. Test login in AWS Console:

- Open AWS Console login page
- Sign in as IAM user with username "loadbalancer" and the password you set
- You should be prompted to change password
- **Save screenshot as:** task4_aws_console_login.png — AWS Console login page with IAM user.



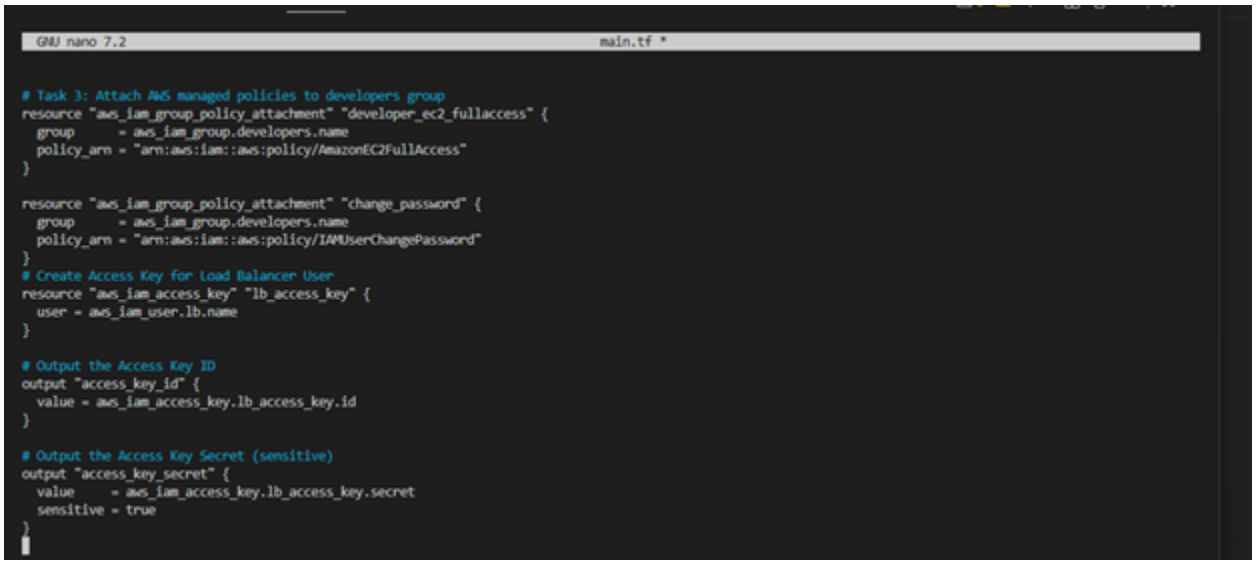
- **Save screenshot as:** task4_aws_console_password_reset.png — Password reset prompt.



Task 5 — Generate Access Keys for IAM User
In this task, you will create access keys for the loadbalancer user and view them in terraform state.
1. Update main.tf to add access key resource and outputs:
Add these resources:

```
resource "aws_iam_access_key" "lb_access_key" {
  user = aws_iam_user.lb.name
}

output "access_key_id" {
  value = aws_iam_access_key.lb_access_key.id
}

output "access_key_secret" {
  value     = aws_iam_access_key.lb_access_key. secret
  sensitive = true
}
```

- **Save screenshot as:** task5_main_tf_access_keys.png — main.tf showing access key resources.



2. Apply the configuration:

terraform apply -auto-approve -var="iam_password=MySecurePass123!"

- **Save screenshot as:** task5_terraform_apply.png — terraform apply output showing access key created.

3. Display outputs:

terraform output

- **Save screenshot as:** task5_terraform_output.png — terraform output showing access_key_id but hidden secret.

```
@Hamail-maker ➜~/Lab13 $ terraform output
access_key_id = "AKIA42E3QACSJKWMHY6M"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSHDTIJ2EOP"
}
user_details = {
  "unique_id" = "AIDA42E3QACSL6WSGHQDT"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@Hamail-maker ➜~/Lab13 $
```

4. View the secret in terraform state:

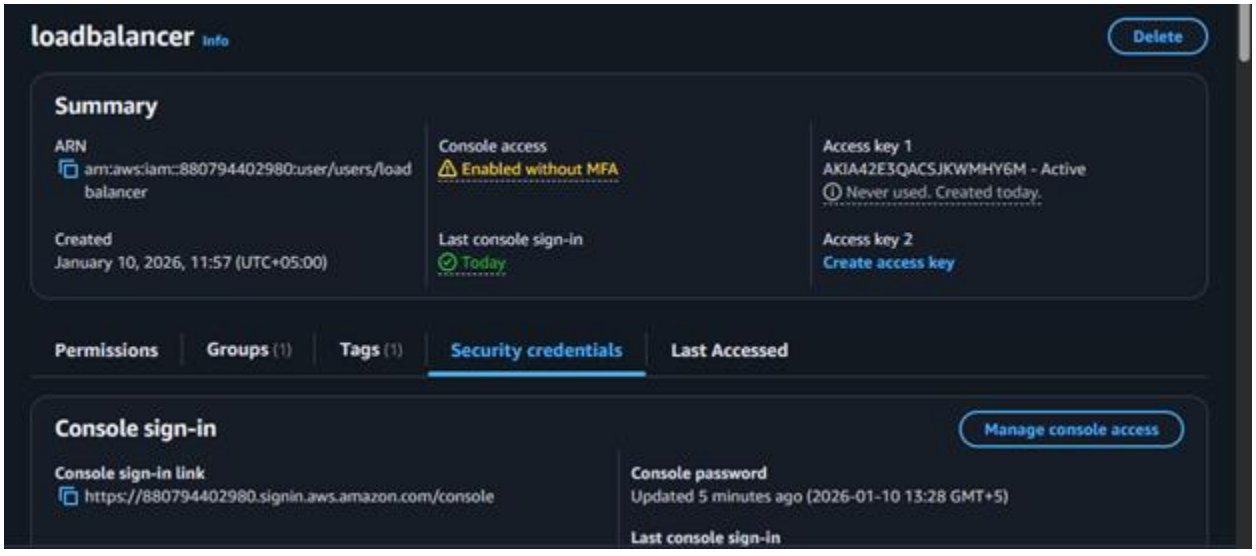
cat terraform.tfstate | grep -A 10 "access_key_secret"

- **Save screenshot as:** task5_tfstate_secret.png — terraform.tfstate showing access key secret.

```
@Hamail-maker ➜~/Lab13 $ cat terraform.tfstate | grep -A 10 "access_key_secret"
    "access_key_secret": {
      "value": "lIKCdlAWAZGS2TlBuPLXonDydEcI6VZZeXk15MdD",
      "type": "string",
      "sensitive": true
    },
    "group_details": {
      "value": {
        "group_arn": "arn:aws:iam::880794402980:group/groups/developers",
        "group_name": "developers",
        "unique_id": "AGPA42E3QACSHDTIJ2EOP"
      },
@Hamail-maker ➜~/Lab13 $
```

5. Verify access key in AWS Console:

- Navigate to IAM → Users → loadbalancer → Security credentials
- **Save screenshot as:** task5_aws_console_access_keys.png — AWS Console showing access keys.



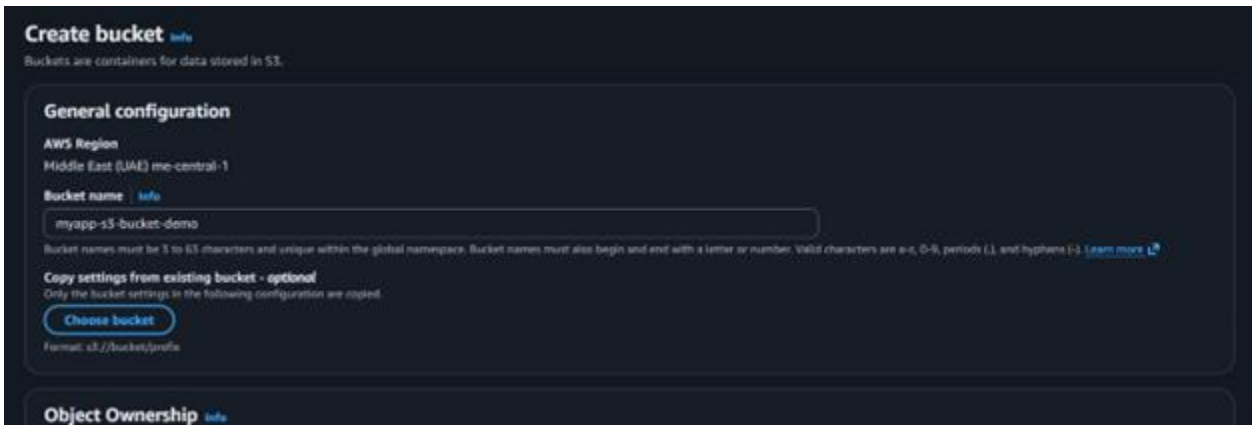
Task 6 — Implement Terraform Remote State with S3

In this task, you will configure Terraform to use S3 backend for remote state storage.

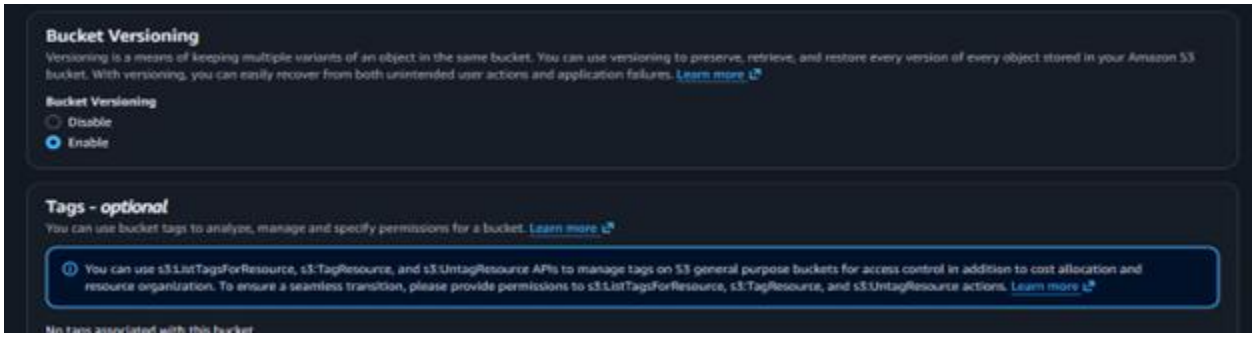
1. Create S3 bucket in AWS Console:

- Navigate to S3 in AWS Console
- Click "Create bucket"
- Bucket name: myapp-s3-bucket-demo (use a unique name if this is taken)

- Enable versioning
- Keep other settings as default
- Click "Create bucket"
- **Save screenshot as:** task6_s3_bucket_create.png — S3 bucket creation page.



- **Save screenshot as:**



— S3 bucket with versioning enabled.

2. Update main.tf to add S3 backend configuration:

Add this at the beginning of main.tf (before the provider block):

```
terraform {
  backend "s3" {
    bucket = "myapp-s3-bucket-demo"
    key    = "myapp/terraform.tfstate"
    region = "me-central-1"
    encrypt = true
    use_lockfile = true
  }
}
```

- **Save screenshot as:**

```
@Hamail-maker ➜~/Lab13 $ cat main.tf
 provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
terraform {
  backend "s3" {
    bucket       = "myapp-s3-bucket-demo-first"
    key          = "myapp/terraform.tfstate"
    region       = "me-central-1"
    encrypt      = true
    use_lockfile = true
  }
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
```

— main.tf showing backend configuration.

3. Reinitialize Terraform with the backend:

terraform init -migrate-state

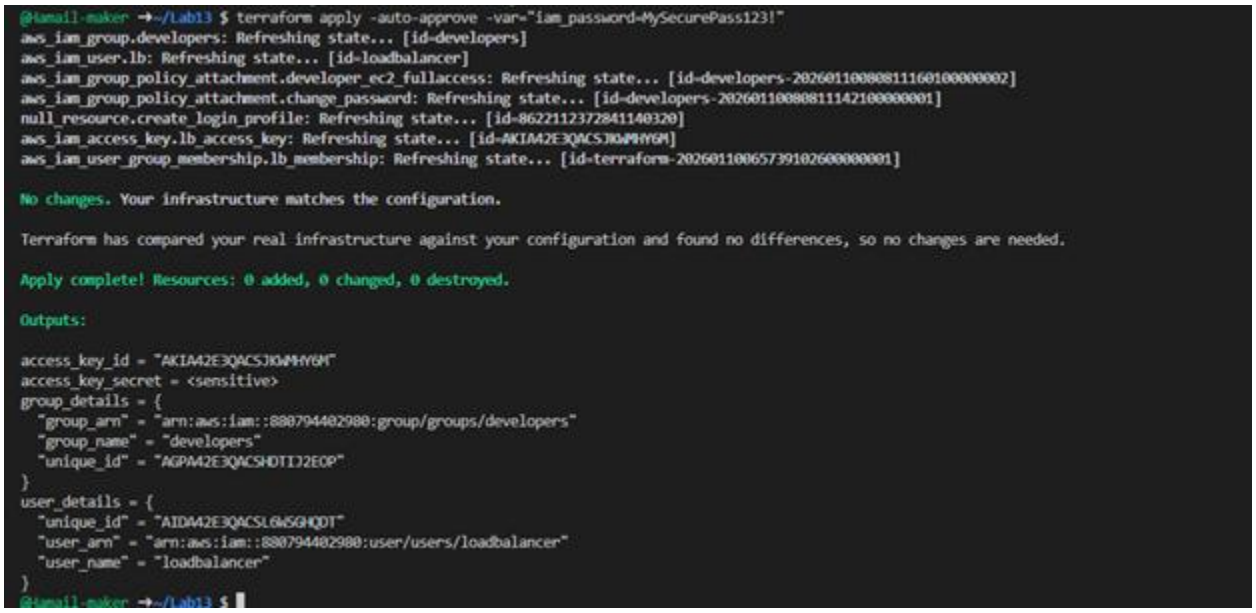
- Type yes when prompted to migrate state
- **Save screenshot as:** task6_terraform_init_migrate.png — terraform init output showing state migration.

```
@Hamail-maker ➜~/Lab13 $ terraform init -migrate-state

Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes


Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/null v3.2.4

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker ➜~/Lab13 $
```

4. Apply the configuration:

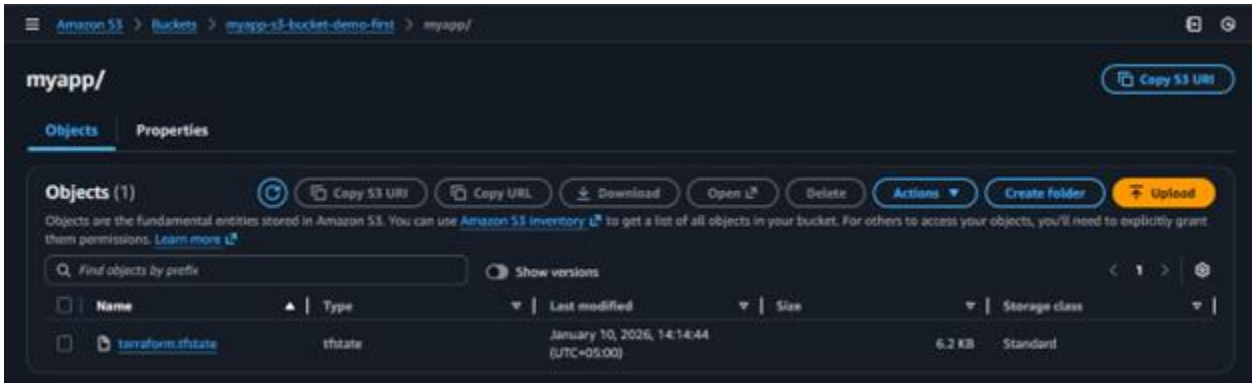
terraform apply -auto-approve -var="iam_password=MySecurePass123!"

- **Save screenshot as:** task6_terraform_apply.png — terraform apply with remote state.



5. Verify state file in S3:

- Navigate to S3 → myapp-s3-bucket-demo → myapp/
- You should see terraform.tfstate file
- **Save screenshot as:** task6_s3_tfstate_file.png — S3 bucket showing terraform.tfstate file.



6. Check local state file:

ls -la terraform.tfstate*

- **Save screenshot as:** task6_local_state_backup.png — showing local state backup file.

7. Destroy resources and verify state change:

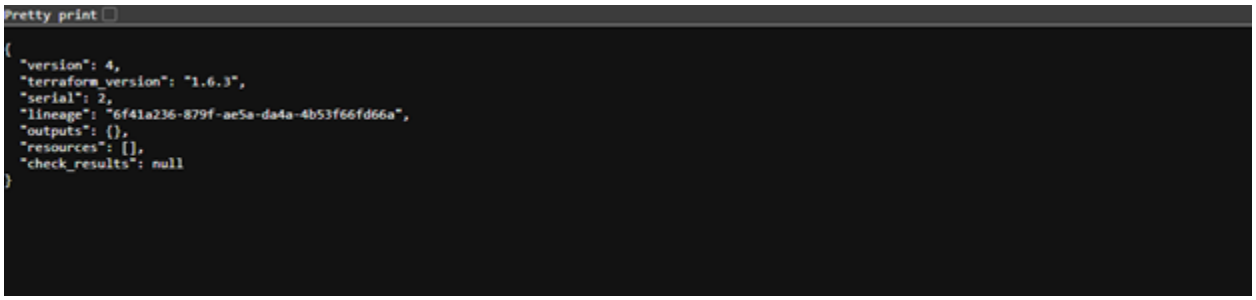
terraform destroy -auto-approve

- **Save screenshot as:** task6_terraform_destroy.png — terraform destroy output.

```
null_resource.create_login_profile: Destroying... [id=8622112372841140320]
null_resource.create_login_profile: Destruction complete after 0s
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destroying... [id=developers-20260110080811160100000002]
aws_iam_group_policy_attachment.change_password: Destroying... [id=developers-20260110080811142100000001]
aws_iam_user_group_membership.lb_membership: Destroying... [id=terraform-20260110065739102600000001]
aws_iam_access_key.lb_access_key: Destroying... [id=AKIA42E3QACSJKWMHYGM]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destruction complete after 1s
aws_iam_user_group_membership.lb_membership: Destruction complete after 1s
aws_iam_group_policy_attachment.change_password: Destruction complete after 1s
aws_iam_group.developers: Destroying... [id=developers]
aws_iam_access_key.lb_access_key: Destruction complete after 1s
aws_iam_user.lb: Destroying... [id=loadbalancer]
aws_iam_group.developers: Destruction complete after 0s
aws_iam_user.lb: Destruction complete after 3s

Destroy complete! Resources: 7 destroyed.
```

8. Verify updated state in S3:

- Refresh S3 bucket view
- Check the terraform.tfstate file (it should show empty resources)
- **Save screenshot as:** task6_s3_tfstate_destroyed.png — S3 state file after destroy.



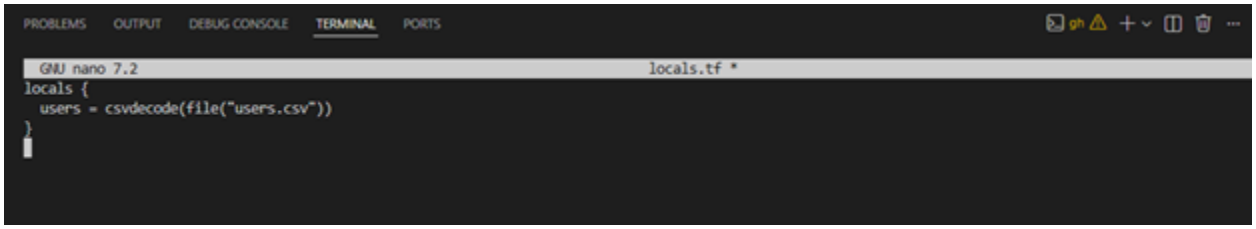
Task 7 — Create Multiple Users from CSV File

In this task, you will create multiple IAM users dynamically from a CSV file.

1. Create locals.tf file:

```
locals {
  users = csvdecode(file("users.csv"))
}
```

- **Save screenshot as:** task7_locals_tf.png — content of locals.tf file.

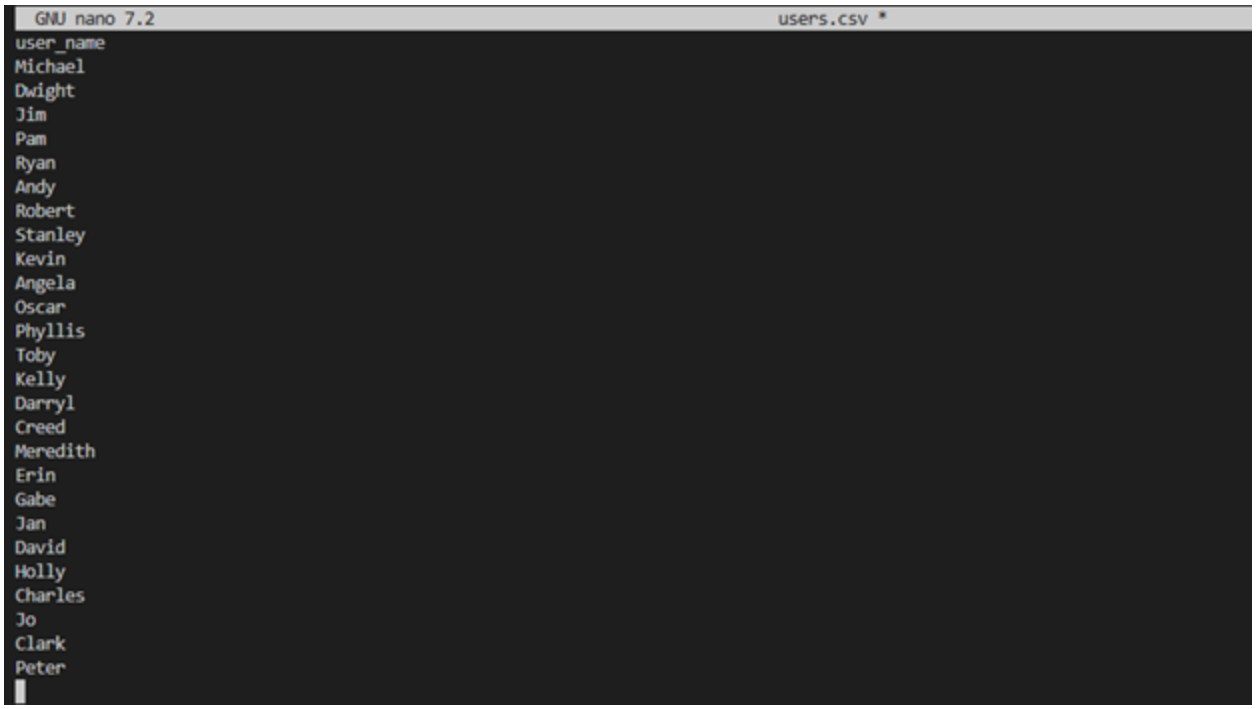


2. Create users.csv file:

```
user_name
Michael
Dwight
Jim
Pam
Ryan
Andy
Robert
Stanley
Kevin
Angela
```

Oscar
Phyllis
Toby
Kelly
Darryl
Creed
Meredith
Erin
Gabe
Jan
David
Holly
Charles
Jo
Clark
Peter

- **Save screenshot as:** task7_users_csv.png — content of users.csv file.



3. Update main.tf to create multiple users:

Replace the single user resources with:

```
# Create multiple IAM users from CSV
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }

  name          = each.value.user_name
  path          = "/users/"
  force_destroy = true

  tags = {
    DisplayName = each.value.user_name
    CreatedBy   = "Terraform"
  }
}
```

```
# Add all users to developers group
resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user = each.value.name
  groups = [
    aws_iam_group.developers.name
  ]
}

# Create login profiles for all users
resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var. iam_password)
    user          = each.value.name
  }

  depends_on = [aws_iam_user. users]

  provisioner "local-exec" {
    command = "${path. module}/create-login-profile. sh ${each.value.name}
'${var.iam_password}'"
  }
}

# Create access keys for all users
resource "aws_iam_access_key" "users_access_keys" {
  for_each = aws_iam_user. users

  user = each.value.name
}

# Output all user details
output "all_users_details" {
  value = {
    for user_name, user in aws_iam_user.users : user_name => {
      user_arn        = user.arn
      user_unique_id   = user.unique_id
      access_key_id    = aws_iam_access_key.users_access_keys[user_name].id
    }
  }
}

# Output all access key secrets (sensitive)
output "all_access_key_secrets" {
  value = {
    for user_name, key in aws_iam_access_key.users_access_keys :  user_name => key.secret
  }
  sensitive = true
}
```

- **Save screenshot as:** task7_main_tf_multiple_users.png — main.tf showing multiple user resources.

```
# --- Task 7: Multiple Users from CSV ---
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }

  name          = each.value.user_name
  path          = "/users/"
  force_destroy = true

  tags = {
    DisplayName = each.value.user_name
    CreatedBy   = "Terraform"
  }
}

resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user   = each.value.name
  groups = [aws_iam_group.developers.name]
}

resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var.iam_password)
    user          = each.value.name
  }
```

4. Reinitialize Terraform (since we changed the configuration significantly):
terraform init

- **Save screenshot as:**

```
@Hamail-maker ➜~/Lab13 $ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/null v3.2.4
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker ➜~/Lab13 $
```

— terraform init output.

5. Apply the configuration to create all users:
terraform apply -auto-approve -var="iam_password=MySecurePass123!"

- **Save screenshot as:** task7_terraform_apply.png — terraform apply output showing all users being created.

```
  "Robert" = {
    "access_key_id" = "AKIA42E3QACSKKH5ND7L"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Robert"
    "user_unique_id" = "AIDA42E3QACSKAFLACDGI"
  }
  "Ryan" = {
    "access_key_id" = "AKIA42E3QACSNO5VM4NX"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Ryan"
    "user_unique_id" = "AIDA42E3QACSNIQWILT43"
  }
  "Stanley" = {
    "access_key_id" = "AKIA42E3QACSCJTEBYNK"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Stanley"
    "user_unique_id" = "AIDA42E3QACSG4GVYNLBM"
  }
  "Toby" = {
    "access_key_id" = "AKIA42E3QACSJLHZM3WW"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Toby"
    "user_unique_id" = "AIDA42E3QACSJGSCHXGGZ"
  }
}
group_details = {
  "group_arn" = "arn:aws:iam::880794402980:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPA42E3QACSFZS5LN52J"
}
user_details = {
  "unique_id" = "AIDA42E3QACSJZRZSW55K"
  "user_arn" = "arn:aws:iam::880794402980:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
```

6. Display the outputs:

terraform output

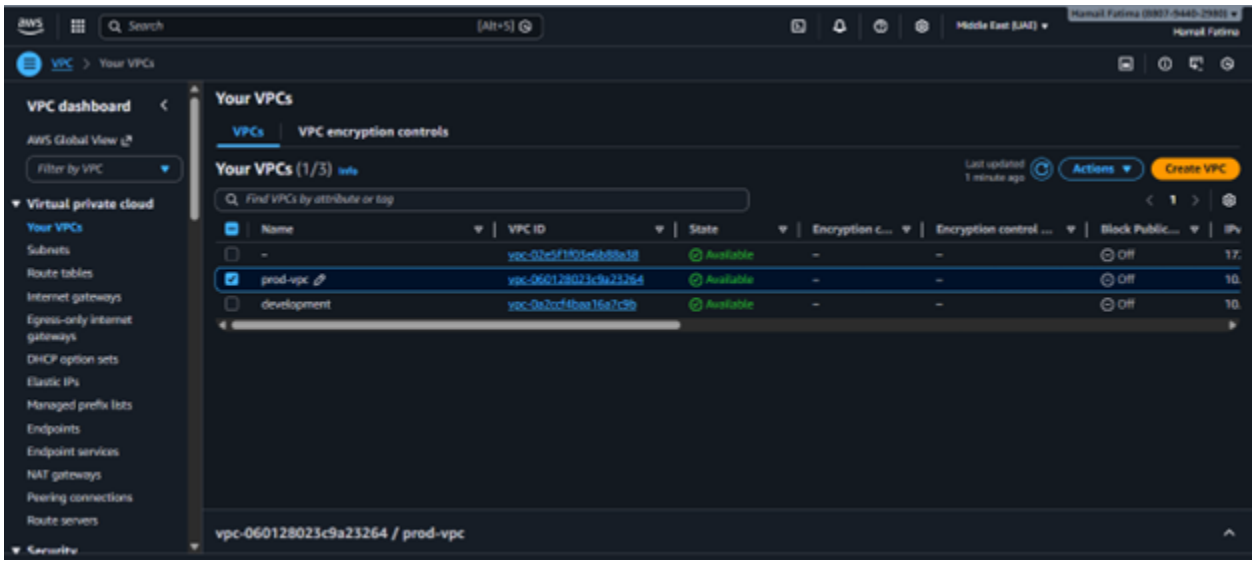
- **Save screenshot as:** task7_terraform_output.png — terraform output showing all user details.

```
@Hamail-maker →~/Lab13 $ terraform output
access_key_id = "AKIA42E3QACSFCJZ7YNJ"
access_key_secret = <sensitive>
all_access_key_secrets = <sensitive>
all_users_details = {
  "Andy" = {
    "access_key_id" = "AKIA42E3QACSG4JFISPJ"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Andy"
    "user_unique_id" = "AIDA42E3QACSJNCLJ2SWB"
  }
  "Angela" = {
    "access_key_id" = "AKIA42E3QACSOJR4QH7H"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Angela"
    "user_unique_id" = "AIDA42E3QACSN2TZWF5VW"
  }
  "Charles" = {
    "access_key_id" = "AKIA42E3QACSPJFUJBPL"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Charles"
    "user_unique_id" = "AIDA42E3QACSBSA2I5RTY"
  }
  "Clark" = {
    "access_key_id" = "AKIA42E3QACSODTP3PMQ"
    "user_arn" = "arn:aws:iam::880794402980:user/users/Clark"
    "user_unique_id" = "AIDA42E3QACSELJGFU3TQ"
  }
```

7. View secrets in terraform. tfstate:

cat terraform.tfstate | grep -A 5 "all_access_key_secrets"

- **Save screenshot as:** task7_tfstate_secrets.png — terraform.tfstate showing access key secrets.

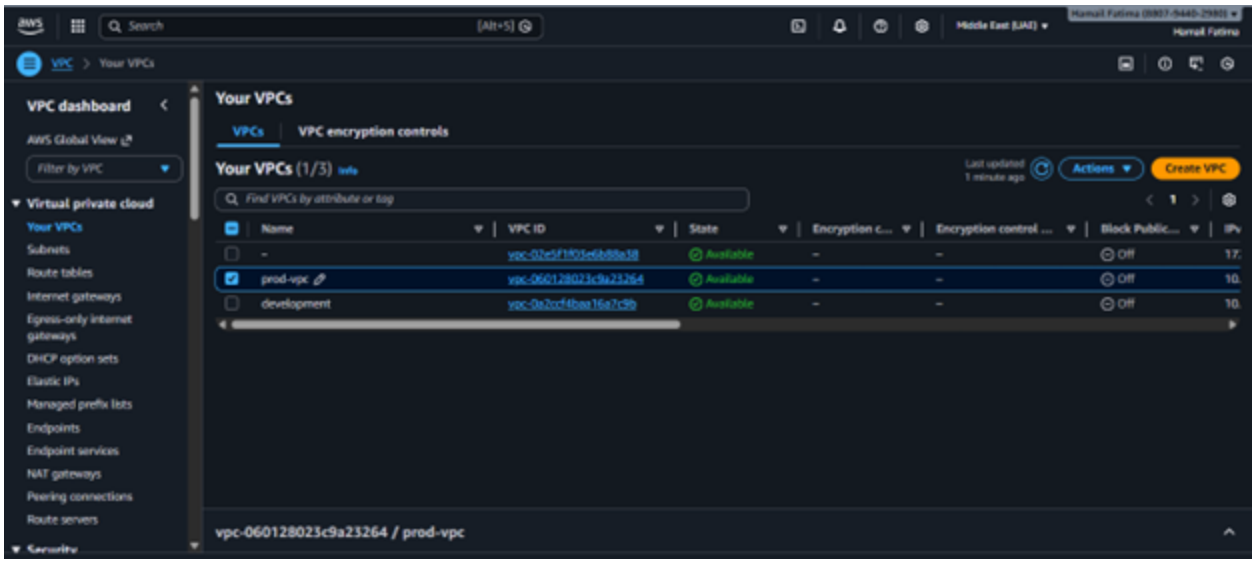


8. Verify all users in AWS Console:

- Navigate to IAM → Users
- **Save screenshot as:** task7_aws_console_all_users.png — AWS Console showing all created users.

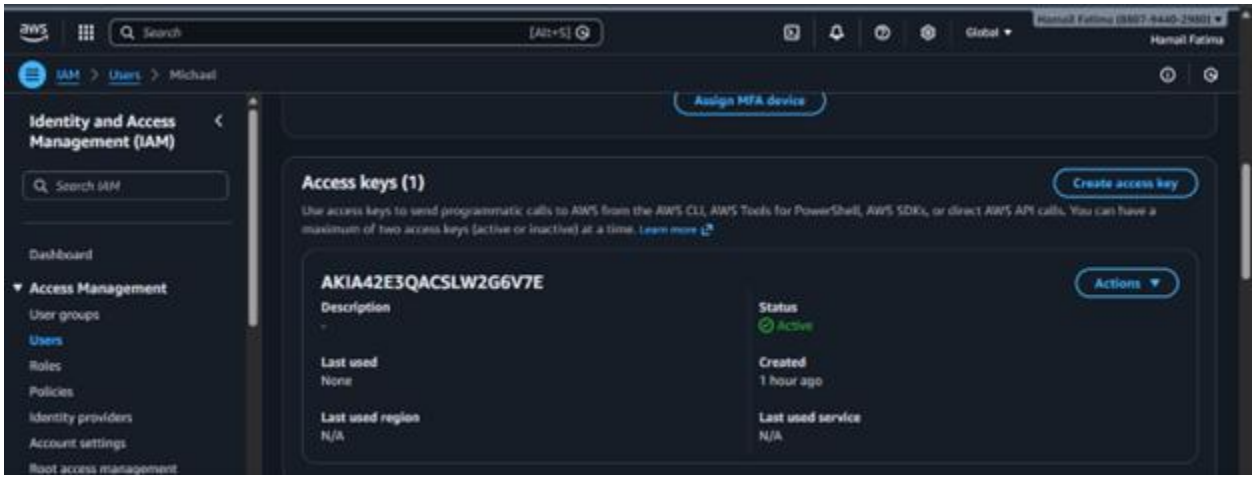
9. Verify group membership:

- Navigate to IAM → Groups → developers → Users tab
- **Save screenshot as:** task7_aws_console_group_members.png — AWS Console showing all users in developers group.



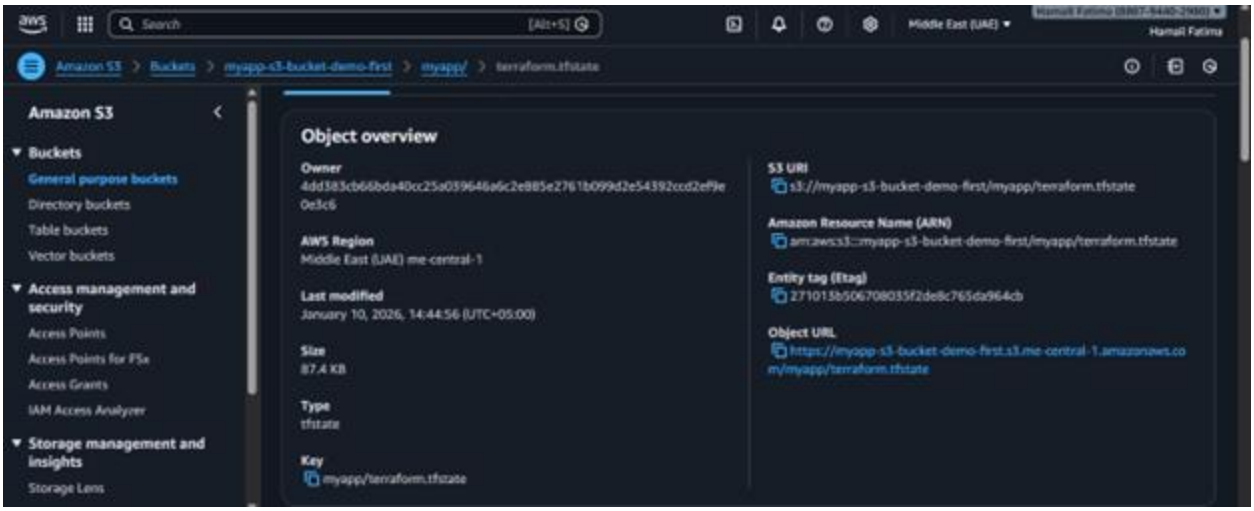
10. Verify one user's access keys:

- Click on any user (e.g., "Michael")
- Go to Security credentials tab
- **Save screenshot as:** task7_aws_console_user_access_key.png — AWS Console showing user's access key.



11. Check terraform state in S3:

- Navigate to S3 bucket and view the state file
- **Save screenshot as:** task7_s3_tfstate_multiple_users.png — S3 showing updated state file.

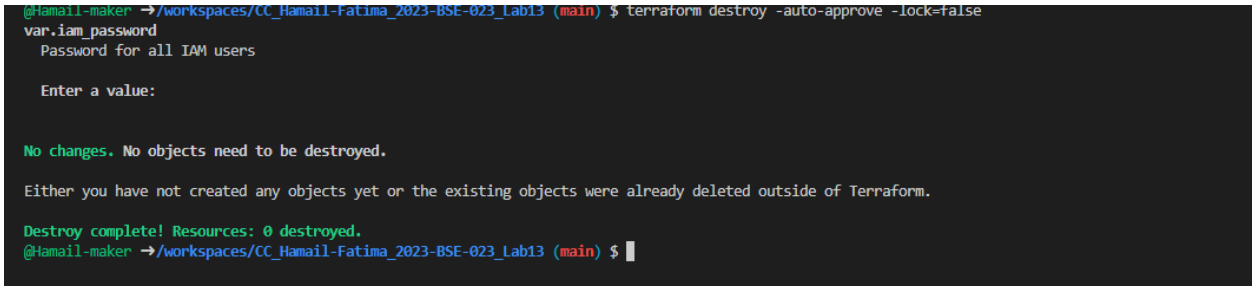


Cleanup

1. Destroy all resources:

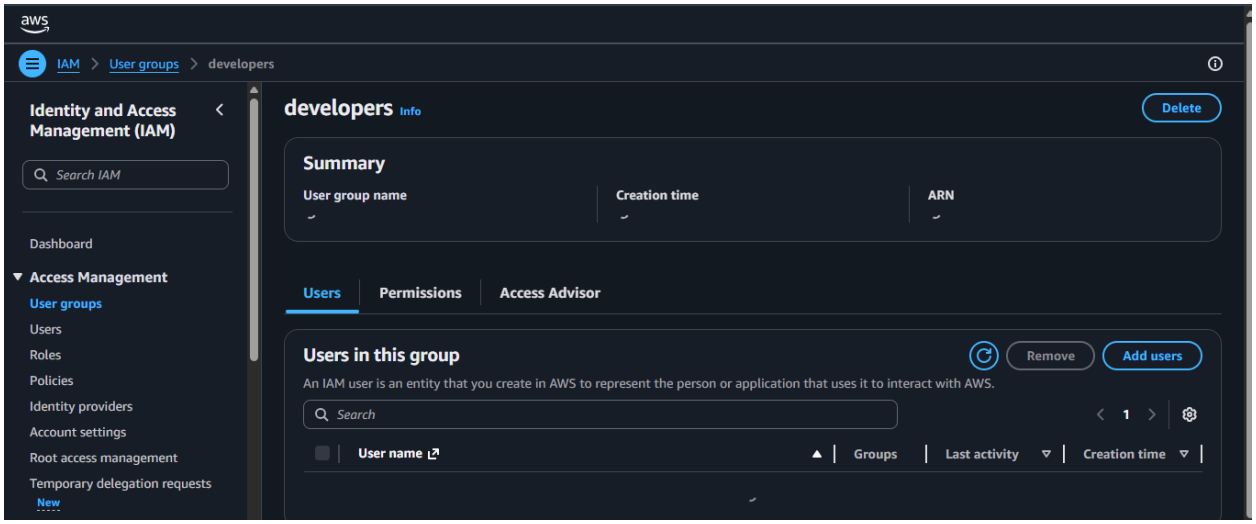
terraform destroy -auto-approve

- **Save screenshot as:** cleanup_destroy_complete.png — terraform destroy completion output.



2. Verify users deleted in AWS Console:

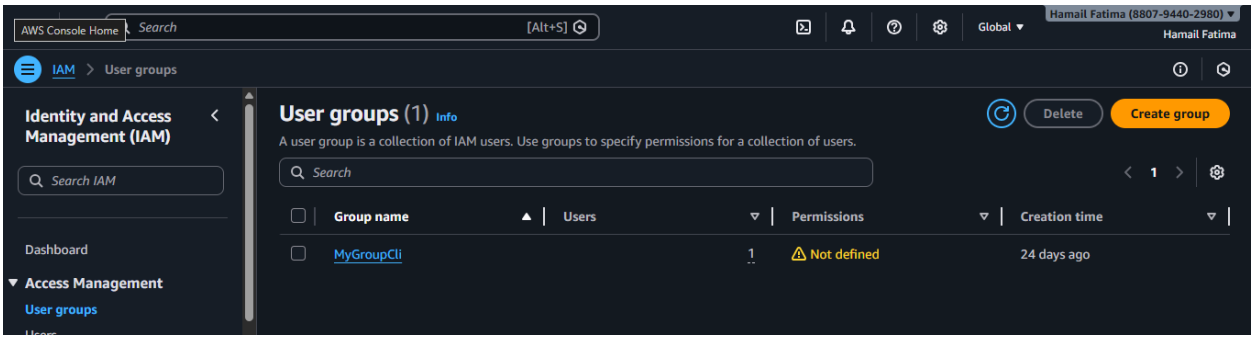
- Navigate to IAM → Users
- **Save screenshot as:** cleanup_aws_console_users_deleted.png — AWS Console showing no users.



3. Verify group deleted in AWS Console:

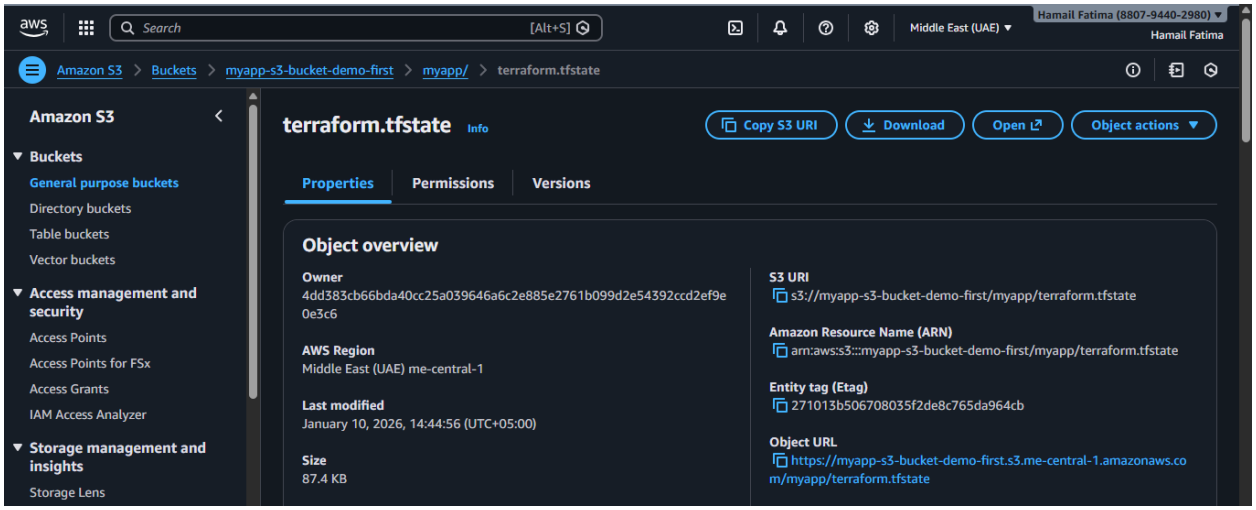
- Navigate to IAM → Groups
- **Save screenshot as:** cleanup_aws_console_group_deleted.png — AWS Console showing

developers group deleted.



4. Check S3 state file:

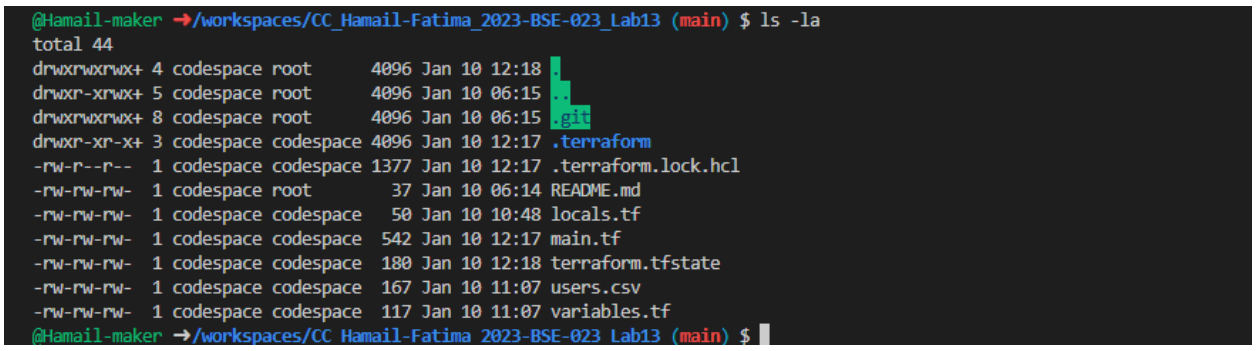
- Navigate to S3 bucket
- **Save screenshot as:** cleanup_s3_empty_state.png — S3 showing empty state.



5. List all project files:

ls -la

- **Save screenshot as:** cleanup_final_files.png — showing final project structure.



6. (Optional) Delete S3 bucket:
   - If you want to clean up completely, delete the S3 bucket from AWS Console
   - **Save screenshot as:** cleanup_s3_bucket_deleted. png — S3 bucket deletion confirmation.